

8. Skriv `fac` och `fib` med hjälp av loopar istället.
9. Vad blir fel om vi inte kopierar `xs` i quicksort?
10. † Pascals triangel ser ut så här:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

osv... Skriv en funktion `pascal(n)` som returnerar den `n`:e raden. Den ska fungera på följande sätt:

```
for i in range(1,8):
    print(pascal(i))
```

```
[1]
[1, 1]
[1, 2, 1]
[1, 3, 3, 1]
[1, 4, 6, 4, 1]
[1, 5, 10, 10, 5, 1]
[1, 6, 15, 20, 15, 6, 1]
```

Kapitel 9

Objektorientering 1: Klasser

9.1 Objektorientering

Objektorientering (förkortas OO) är ett programmeringsparadigm eller programmeringsstil där man samlar data och datastrukturer med de funktioner som kan tillämpas på dem. I Python är i princip allt implementerat som objekt. Detta gäller såväl heltal, flyttal, strängar, listor, uppslagstabeller som funktioner. Vad menas då med att knyta ihop datastrukturer med “funktioner som kan tillämpas på dem”? Ta till exempel listor:

```
l = [2,7,2]
print(l.append(1))
```

```
[2, 7, 2, 1]
```

I objektorienterade termer har vi på första raden skapat ett objekt `l` av typ `list`. Vi har på andra raden använt funktionen `append` som är *specifik* för datatypen `list`. I objektorientering benämner man funktioner som är knutna till objekt för *metoder*. De anropas med hjälp av punktnotationen `object.method(<parameters>)`. Det är en notation som du vid det här laget kanske känner igen? Ta till exempel strängar och operationer vi gör på dem:

```
s = 'Hej'
print(s.lower())
```

```
hej
```

Här har vi skapat ett objekt `s` av typ `str` och använt metoden `lower` (specifik för datatypen `str`).

9.1.1 Varför objektorientering?

Objektorientering lämpar sig bra att använda när man vill:

- Knyta samman mycket data som relaterar till varandra (hör till samma objekt).
- Kunna definiera egna datatyper.

Säg till exempel att vi vill hålla data associerat till en bil (kanske för ett datorspel) i minnet i ett program. Det kan vara däckbredd, färg, antal dörrar, nuvarande bensinnivå, hastighet, vinkel på framhjulen och

en mängd annan data. Det skulle då vara smidigt att ha all data för denna bil “på ett ställe”. Om vi hade ett bil-objekt `b` skulle vi komma åt och kunna ändra alla *attribut* i detta objekt med

```
b.tire_width()
b.color()
b.increase_speed(2)
```

Det är naturligt att tänka på objekt som “fysiska ting” (t.ex. en bil), men det finns även naturliga objekt som vi inte kan ta på. Vi kommer strax visa hur vi implementerar en kurs (t.ex. DA2005) där vi kan samla användarinformation, resultat, med mera. Slutligen ska vi även titta på hur vi kan implementera vår egen datastruktur för att representera heltalsmängder.

Två fördelar med att skriva objektorienterad kod är:

1. Det uppmuntrar god struktur genom att samla data och relaterad funktionalitet på samma ställe.
2. Det är bra för abstraktion, eftersom det blir lättare att gömma detaljer.

9.1.2 Hur skapas objekt?

Hur skriver vi då objektorienterad kod? För att skapa egna objekt (tänk till exempel på en bil eller en kurs som ett objekt) så måste vi ha en specifikation för objektets data, metoder och allmänna struktur. En sådan specifikation heter *klass* i objektorienteringens terminologi. Vi kan skapa en klass som paketerar data och funktionalitet (du kan tolka “klass” som “klassificering”, som minnestrick). En klass är att jämföra med en typ och ett objekt är en *instans* av en klass. Till exempel är `[1, 2, 3]` en instans av klassen `list`. Varje instans kan ha olika *attribut* kopplade till den för att representera dess nuvarande tillstånd. Ett sätt att förstå attribut är att tänka på dem som ett objekts “egna” variabler. Instanser kan även ha *metoder*, att tänka på som funktioner definierade i klassen, för att modifiera dess tillstånd. Ett exempel på en metod för listor är `pop()`, som man kan tillämpa på ett objekt `xs` med `xs.pop()`.

Vi har i kursen använt många exempel på klasser: heltal, listor, uppslagstabeller, med mera. Vi ska nu lära oss att skriva våra egna.

9.1.3 Att definiera en klass

Ett minimalt exempel är för en tom klass:

```
class Course:
    pass
```

Vi har döpt klassen till `Course`. Konventionen är att klassnamn börjar med stor bokstav.

Klassdefinitionen ovan gör det möjligt att *instansiera* klassen till *objekt*. Man kan sedan knyta olika *attribut* till objektet.

```
k = Course()          # An object is created, an instance of the class Course
k.code = 'DA2005'     # An attribute is assigned to the object
k2 = Course()         # A new object is created
print(k.code)
```

```
DA2005
```

Om man försöker använda attributet `code` för objektet `k2` så får man ett felmeddelande eftersom vi inte har tillskrivit `k2` något sådant attribut:

```
print(k2.code) # This object does not have an attribute "code"
```

```
AttributeError: 'Course' object has no attribute 'code'
```

Här är `code` alltså en variabel som tillhör objektet/instansen `k`. Vi kommer ofta säga “instansattribut” för denna typ av variabler.

9.1.4 Metoder

En *metod* är en funktion definierad i en klass. I `Course` (nedan) är alla **def**-satser metoder. När man definierar metoder följer man samma konventioner som för funktioner, det vill säga identifierarna skrivs med små bokstäver och om man använder flera ord så separeras de med `_`.

```
class Course:

    def __init__(self, code, name, year=2024):          # Constructor
        self.participants = []
        self.code = code
        self.name = name
        self.year = year

    def number_participants(self):
        return len(self.participants)

    def new_participant(self, name, email):
        self.participants.append((name, email))
```

Ignorera identifieraren `self` en liten stund.

Den första metoden, `__init__`, har en särställning och kallas *konstruktor*. En konstruktor är den metod som initierar ett nytt objekt. Konstruktorn anropas bara vid instansiering (alltså då vi skapar objektet). Metoder med identifierare på formen `__fkn__` används av Python-systemet, så man bör vara försiktig med hur dessa används. Konstruktorn heter alltid `__init__`, så när vi kör

```
da2005 = Course('DA2005', 'Programmeringsteknik')
```

anropas konstruktormetoden `Course.__init__` och resultatet sparas i variabeln `da2005`. Objektet i `da2005` är då en *instans* av `Course`-klassen. Det är vidare endast `__init__` som anropas vid instansiering. De övriga metoderna finns tillgängliga att använda när vi anropar dem.

Vi kan modifiera ett objekt genom att anropa dess metoder. Detta gör man genom punkt-notation (precis som vi använt `xs.pop()` och andra listmetoder för att modifiera listor).

```
da2005.new_participant('Johan Jansson', 'j.jansson@exempel.se')
print(da2005.number_participants())
```

```
1
```

9.1.5 Self

Vad är då uttrycket `self` som finns med framför alla attribut och som parametrar till metoder? Variabeln `self` i `Course`-klassen representerar objektet som metoden anropas från (man måste inte kalla det argumentet för `self`, men det är en konvention bland Python-programmerare att göra så). I exemplet

ovan refererar `self` i `new_participant` till objektet `da2005`. Detta kan vara svårt att greppa när man först lär sig om klasser och objektorientering. Som illustration kan du tänka dig att raden

```
da2005.new_participant('Johan Jansson', 'j.jansson@exempel.se')
```

egentligen motsvarar:

```
Course.new_participant(da2005, 'Johan Jansson', 'j.jansson@exempel.se')
```

dvs att vi plockar fram metoden `new_participant` från klassen `Course` och i första parametern anger vilket objekt som ska användas.

Obs: Detta är inte korrekt syntax i Python, utan endast en illustration av konceptet att metoden `new_participant` tar objektet `da2005` som parameter för att kunna modifiera det.

9.1.6 Klass- och instansattribut

Attribut är ett annat sätt att benämna variabler som är specifika för objekt eller klasser. Det finns två olika typer av attribut; *instansattribut* och *klassattribut*. Generellt sett används *instansattribut* för att hålla data som är unik för varje instans (dvs, specifika för objektet) och *klassattribut* används för attribut och metoder som delas av alla instanser av klassen:

```
class Course:

    university = 'Stockholm University' # class attribute shared by all instances
                                         # (1) Occurs outside of constructor
                                         # (2) Has no 'self', i.e., specific object,
                                         # associated to it

    def __init__(self, code, name, year=2024):
        # instance attributes below, unique to each instance
        self.participants = []
        self.code = code
        self.name = name
        self.year = year
```

Exempelanvändning:

```
da2005 = Course('DA2005', 'Programmeringsteknik')
da2005 = Course('DA4007', 'Programmeringsteknik II', year = 2025)

print(da2005.university)           # shared by all courses
print(da4007.university)           # shared by all courses
print(da2005.code, da2005.year, da2005.name) # unique to da2005
print(da4007.code, da4007.year, da4007.name) # unique to da4007
print(Course.university)           # you can also access a class
                                     # attribute from the class itself
```

```
Stockholm University
Stockholm University
DA2005 2024 Programmeringsteknik
DA4007 2025 Programmeringsteknik II
Stockholm University
```

Vi kan även skriva över nuvarande värden på attribut genom

```
da2005.year = 2025
print(da2005.year)
```

```
2025
```

På samma sätt som vi ovan skrivit över instansattributet `year` i `da2005`, men inte i `da4007`, kan vi även skriva över metoder i ett objekt (kom ihåg att funktioner ej har någon särställning utan är som andra objekt i Python).

Varning: Muterbara data kan ha överraskande effekter om de används som klassattribut!

Om vi till exempel hade lagt listan `participants` som en klassvariabel kommer den delas av *alla* kurser:

```
class Course:

    university = 'Stockholm University' # class attribute shared by all instances
    participants = []                  # class attribute shared by all instances

    def __init__(self, code, name, year=2024):
        # instance attributes below, unique to each instance
        self.code = code
        self.name = name
        self.year = year

    def new_participant(self, name, email):
        self.participants.append((name, email))

d = Course('da2005', 'Programmeringsteknik')
e = Course('da4007', 'Programmeringsteknik (online version)')

d.new_participant('Johan Jansson', 'j.jansson@exempel.se')
print(d.participants) # shared by all courses
print(e.participants) # shared by all courses

[('Johan Jansson', 'j.jansson@exempel.se')]
[('Johan Jansson', 'j.jansson@exempel.se')]
```

9.1.7 Privata och publika metoder och attribut

I objektorienterad programmering skiljer man ofta på *privata* och *publika* metoder och attribut. De publika metoderna utgör *gränssnittet* (eng. *interface*) för en klass och är de metoder som användare av klassen ska använda. De privata metoderna och attributen är för internt bruk. I många programmeringsspråk finns det stöd för att kontrollera åtkomst av metoder och attribut så att man inte av misstag börjar förlita sig på annat än klassens gränssnitt. På detta sätt uppnår man *abstraktion*¹ med hjälp av objektorientering:

Python fokuserar på enkelhet och har istället en konvention: identifierare för attribut/metoder som börjar med understreck `_` ses som *privata*. De bör alltså bara användas internt.

Vi har nedan lagt till en privat metod `_check_duplicate` i vårt kursexempel. Det är vanligt att studenter registrerar sig med olika email-adresser. Metoden kontrollerar om identiska namn förekommer i del-

¹Wikipedia: [Abstraction in object oriented programming](#).

tagarlistan när vi lägger till en deltagare och skriver i så fall ut en varning. Vi kan då vidta lämpliga åtgärder (t.ex. implementera en metod `remove_participant` för att ta bort dubbla deltagare).

```
import warnings # included in Python's standard library

class Course:

    def __init__(self, code, name, year=2024): # Constructor
        self.participants = []
        self.code = code
        self.name = name
        self.year = year

    def _check_duplicate(self, name, email):
        for p in self.participants:
            if name == p[0]: # check if identical name
                # raise warning and print message if identical name:
                # uses string formatting, see e.g.:
                # https://www.w3schools.com/python/ref_string_format.asp
                warnings.warn(
                    'Warning: Name already exists under entry: {0}'.format(p))

    def number_participants(self):
        return len(self.participants)

    def new_participant(self, name, email):
        self._check_duplicate(name, email)
        self.participants.append((name, email))
```

Vi kan nu se vad som händer om vi registrerar studenter med samma namn:

```
da2005 = Course('DA2005', 'Programmingsteknik')
da2005.new_participant('Johan Jansson', 'j.jansson@exempel.se')
da2005.new_participant('Johan Jansson', 'j.jansson@gmail.com')
```

Den tredje raden kommer att generera en “varning” som skrivs ut

```
UserWarning: Warning: Name already exists under entry:
('Johan Jansson', 'j.jansson@exempel.se')
```

I detta exempel används metoden `_check_duplicate` för internt bruk i klassen `Course`. Det är alltså inte tänkt att användaren av klassen ska anropa denna metod. Metoden är inte med i *gränssnittet* och ökar således inte storleken (komplexiteten i användning) av vår klass. Syftet med privata metoder är att kunna definiera metoder som är praktiska att använda internt i klassen, men som man vet kan vara olämpliga eller ologiska som gränssnittsmetod. I exemplet ovan är det en praktiskt hjälpmetod med användbar funktionalitet när vi lägger till deltagare. Notera att även om det är markerat med `_` att metoden är privat, så kan vi fortfarande anropa den med följande explicita metoanrop,

```
da2005._check_duplicate('Johan Jansson', 'j.jansson@exempel.se')
```

men det betraktas som dålig programmering.

9.1.8 Ordlista

- *Objekt*: ett element av någon typ med olika attribut och metoder.
- *Klass*: en typ innehållande attribut och metoder (t.ex. `list`, `dict`, `Course` från exemplet ovan, etc.).
- *Instansiering*: då ett objekt skapas från en viss klass.
- *Metod*: en funktion definierad i en klass (ex. `pop`, `insert`).
- *Konstruktör*: metod för att instansiera ett objekt av klassen (ex. `dict()`). Varje klass har en konstruktör `__init__` som antingen är implicit eller explicit definierad.
- *Klassattribut*: variabel definierad utanför konstruktorn i en klass (delas av alla instanser av klassen).
- *Instansattribut*: variabel definierad i konstruktorn för klassen (eller satt med `self` i klassen). Tillhör en specifik instans av klassen.

9.2 Modularitet genom objektorientering

I exemplet `Course` ovan har vi använt en lista med tupler för att representera deltagarlistan `participants`. En mer objektorienterad lösning hade varit att ha en separat klass för deltagare:

```
class Participant:

    def __init__(self, name, email):
        self._name = name
        self._email = email

    def name(self):
        return self._name

    def email(self):
        return self._email
```

Vi måste nu modifiera metoderna i `Course` så att de hanterar deltagare representerade som `Participant` istället för tupler av strängar. Exempelvis:

```
class Course:

    # old code unchanged, except for _check_duplicate which
    # has to be modified to take a Participant

    def new_participant(self, p):
        self._check_duplicate(self, p)
        self.participants.append(p)
```

Vi måste nu skapa deltagare genom:

```
d = Participant('Johan Jansson', 'j.jansson@exempel.se')
da2005.new_participant(d)
da2005.new_participant(Participant('Foo Bar', 'foo@bar.com'))
```

Det kan tyckas konstigt att skriva om sin kod så här, men fördelen är att eventuella framtida förändringar kommer bli mindre komplexa. Exempelvis, låt oss säga att vi vill separera förnamn och efternamn för deltagare. Då räcker det att skriva:


```

class Participant:

    def __init__(self, fname, lname, email):      # Changed!
        self._fname = fname                      # Changed!
        self._lname = lname                      # Changed!
        self._email = email

    def name(self):
        return self._fname + ' ' + self._lname  # Changed!

    def email(self):
        return self._email

```

Koden för Course behöver inte alls ändras eftersom den bara använder Participant-gränssnittet. På detta sätt leder objektorientering till mer modular kod, dvs varje klass kan ses som en liten kodmodul i sig och interna modifikationer påverkar ingen annan kod i programmet så länge gränssnittet är oförändrat.

9.3 Ett till exempel på objektorientering: talmängder

Python har en typ för mängder, liknande uppslagstabeller, men nycklarna är inte associerade med några värden. För dokumentation se:

<https://docs.python.org/3/tutorial/datastructures.html#sets>

<https://docs.python.org/3/library/stdtypes.html#set-types-set-frozenset>

Syntaxen för att skapa en mängd (**set**) liknar den för listor, men med {} istället för []. Några exempel:

```

basket = set(['apple', 'banana', 'orange', 'pear'])
# the above is equivalent to
basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}
print(basket)
print('orange' in basket)
print('crabgrass' in basket)

a = set([1,2,5])
b = set([3,5])
print(a)
print(a - b) # difference
print(a | b) # union
print(a & b) # intersection
print(a ^ b) # numbers in a or b but not both

```

```

{'pear', 'apple', 'orange', 'banana'}
True
False
{1, 2, 5}
{1, 2}
{1, 2, 3, 5}
{5}
{1, 2, 3}

```

9.3.1 Egen implementation

Här kommer ett till större exempel taget från kursboken (Introduction to Computation and Programming using Python av John V Guttag, sida 111), där vi implementerar vår egen mängd-datatyp `IntSet` med hjälp av listor. Vi kan representera mängder av heltal som listor, med invarianten (en garanterad egenskap) att det inte finns några dubletter.

```
class IntSet:
    '''An IntSet is a set of integers'''

    def __init__(self):
        '''Create an empty set of integers'''
        self.vals = []

    def insert(self,e):
        '''Assumes e is an integer and inserts e into self'''
        if e not in self.vals:
            self.vals.append(e)

    def member(self,e):
        '''Assumes e is an integer.
        Returns True if e is in self, and False otherwise'''
        return e in self.vals

    def delete(self,e):
        '''Assumes e is an integer and removes e from self
        Raises ValueError if e is not in self'''
        try:
            self.vals.remove(e)
        except:
            raise ValueError(str(e) + ' not found')

    def get_members(self):
        '''Returns a list containing the elements of self.
        Nothing can be assumed about the order of the elements'''
        return self.vals
```

Vi bibehåller *invarianten* att ett `IntSet` ej innehåller några dubletter i alla funktioner.

Lite tester:

```
x = IntSet()
print(x)
x.insert(2)
x.insert(3)
x.insert(2)
print(x)
print(x.member(5))
print(x.member(3))
```

```
x.delete(2)
print(x.get_members())
```

```
<__main__.IntSet object at 0x7febb7002040>
<__main__.IntSet object at 0x7febb7002040>
False
True
[3]
```

De två första print-anropen skriver ut information om objektet; var det är definierat och på vilken minnesadress det sparats på. Om man kör denna kodsnuitt igen kommer troligtvis objektet ligga på ett annat ställe i minnet.

9.3.2 Integrera din klass i Python-systemet

Visst vore det bra att kunna lägga till egna datastrukturer som är lika integrerade i Python-systemet som list, dict, set, och så vidare? Det är möjligt och till och med att betrakta som god programmering. Det man skulle behöva är att metoder som Python tillåter på inbyggda objekt/klasser också är tillämpbara på dina egna klasser. Här beskriver vi hur.

9.3.2.1 str, len och andra anpassningar

Vi kan lägga till metoder `__str__(self)` och `__len__(self)` som gör att vi kan använda oss av Python-systemets syntax `str()`, och `len()`. På detta sätt kan vi få mängder att skrivas ut snyggare än ovan och ett exempel på hur `__str__` metoden kan se ut är

```
def __str__(self):
    result = ''
    for e in self.vals:
        result += str(e) + ', '
    return '{' + result[:-1] + '}'
```

Detta gör att följande skrivs ut istället:

```
x = IntSet()
print(x)
x.insert(2)
x.insert(3)
x.insert(2)
print(x)
```

```
{ }
{2,3}
```

9.3.2.2 Integrering av jämförelseoperatorer

Låt oss säga att vi vill kunna jämföra `IntSet` med `<=` och att `x <= y` ska tolkas som att "x är en delmängd till y". Då kan vi lägga till följande metod till `IntSet` klassen:

```
def __le__(self, other):
    for x in self.vals:
        if not other.member(x):
```

```
        return False
    return True
```

Med tester:

```
print(x <= x)
empty = IntSet()
print(empty <= x)
print(x <= empty)
```

```
True
True
False
```

Vi kan på detta sätt definiera de olika standardoperationerna i Python (+, <, =, ...) för våra egna klasser. Vi kan även *operatoröverlagra* andra operatörer som **in**, **and** med flera. För dokumentation kring namn på standardoperationer för metoder se <https://docs.python.org/3/library/operator.html>. Till exempel kan vi döpa om metoden `member` i vår `IntSet`-klass till `__contains__`. Då kan vi använda oss av operatören **in** och skriva `5 in x` istället för att skriva `x.member(5)`.

9.4 Uppgifter

1. Utöka klasserna `Course` och `Participant` med metoden `__str__` så att vi kan använda oss av `str()` för att få en fin textrepresentation av objekt från de klasserna. Du kan använda dig av exempelkoden nedan (i avsnitt 9.4.1). Det är valfritt hur strängen ser ut, men kurskod och kursnamn ska synas för `Course`-objekt och namn och email ska förekomma i `Participant`-objekt.
2. Lägg till attributen `teacher` av typ `bool` till klassen `Participant`. Skriv ut lärarens namn på något speciellt sätt så att man från `Participant`-listan ser vem det är.
3. Lägg till metoder så att operatörerna `==` och `!=` finns för klassen `IntSet`. Operatören `==` ska returnera **True** om mängderna innehåller samma element, annars **False**. Operatören `!=` är negationen (logiska motsatsen) av `==`.

Tips: läs dokumentationen <https://docs.python.org/3/library/operator.html>.

4. Implementera union, differens och snitt för `IntSet`. Snitt kan du implementera på valfritt sätt, men implementera union genom att operatoröverlagra `+` och differens genom att operatoröverlagra `-`. Metoderna skall returnera ett nytt `IntSet` objekt. Till exempel:

```
x = IntSet()
x.insert(2)
x.insert(3)
x.insert(1)

y = IntSet()
y.insert(2)
y.insert(3)
y.insert(4)
print(x == y, y != x)
print(x.intersection(y))
print(x + y, y - x)
```

```
False True
{2,3}
{4,2,3,1} {4}
```

5. Implementera en klass Dog med funktionalitet som möjliggör gränssnittet nedan.

```
d = Dog('Fido')
e = Dog('Buddy')
d.add_trick('roll over')
e.add_trick('play dead')
d.add_trick('shake hands')
print(d.name, d.tricks)
print(e.name, e.tricks)
```

```
Fido ['roll over', 'shake hands']
Buddy ['play dead']
```

6. Implementera metoderna `__str__` och `__gt__` i klassen Dog så vi kan skriva `str(d)` och jämföra `d > e` (> baseras på hur många trick de kan). Exempel nedan (givet koden vi körde i uppgift 5)

```
print(str(d))
print(d > e)
```

```
Fido knows: roll over, shake hands
True
```

9.4.1 Exempelkod för uppgifterna

```
import warnings # included in Python's standard library

class Course:

    def __init__(self, code, name, year=2024): # Constructor
        self.participants = []
        self.code = code
        self.name = name
        self.year = year

    def _check_duplicate(self, p):
        for other in self.participants:
            if p.name() == other.name(): # check if identical name
                # raise warning and print message if identical name:
                warnings.warn(f'Name already exists under entry:' + str(p))

    def number_participants(self):
        return len(self.participants)

    def new_participant(self, p):
        self._check_duplicate(p)
        self.participants.append(p)
```

```
class Participant:

    def __init__(self, fname, lname, email):
        self._fname = fname
        self._lname = lname
        self._email = email

    def name(self):
        return self._fname + ' ' + self._lname

    def email(self):
        return self._email
```